SOFTWARE DETAILED DESIGN

Project IV

ABSTRACT

DocNest is a secure client–server document storage system that allows authenticated users to upload, download, list, and delete documents stored on a centralized server. The system consists of a JavaFX client application and a console-based Java server communicating through TCP sockets using a custom binary packet protocol. The server manages user authentication, file storage within per-user vault directories, metadata management using JSON files, and event logging. The client provides a graphical interface for interacting with stored files and issuing commands to the server. The design follows an object-oriented architecture and implements a state-driven protocol to ensure reliable communication and secure operation.

Team Members

Moutushi Sarkar

Table of Contents

System Level Design 3
- Description 3
- System Architecture Description 3
  - System Architecture Diagram 4
  - Data Packet Structure 4
  - Packet Layout Diagram 5
  - Command Protocol 5
  - Server Lifecycle State Machine 6
  - Server Lifecycle Diagram 6
  - Session Protocol State Machine 6
  - Allowed Commands per State 7
  - Session Protocol State Machine Diagram 7
  - Communication Flow 7

Client Design 10
- Description 10
- Client Software Architecture Description 10
  - Client Architecture Diagram 10
  - Client Class Diagram 11
  - ClientSessionManager – Moutushi Sarkar 11
  - PacketBuilder – Moutushi Sarkar 12
  - Client Data Flow 13
- Graphical User Interface Design Description 14

Server Design 17
- Description 17
- Server Software Architecture Description 17
- User Interface Design Description 19
  - Example Server Console Output 19
- Server Modules 20
  - ServerMain – Moutushi Sarkar 20
  - ClientSession – Moutushi Sarkar 20
  - PacketRouter - Moutushi Sarkar 20
  - MetadataStore - Moutushi Sarkar 21

FileStorageManager - Moutushi Sarkar 21
Logger - Moutushi Sarkar 21
Data Structures 22
FileRecord 22
UserRecord 22
DataPacket Class 22
Core Algorithms 23
Authentication Process 23
File Upload Process 23
File Download Process 24
File Deletion Process 24
Error Handling 25
Error Packet Format 25
Error Categories 25
Error Logging 25
Security Design 26
Credential Storage 26
Vault Isolation 26
Session Enforcement 26
Traceability to Requirements 26

System Level Design

This document describes the detailed software design of the DocNest system. It defines the architecture, modules, data structures, communication protocol, and operational workflows required to implement the system described in the Software Requirements Specification (SRS).

Description

DocNest is a distributed client–server system designed to securely store and manage user documents over a network. The system allows authenticated users to upload, download, list, and delete files that are stored on a central server.

The system consists of two main applications:

1. **JavaFX Client Application**
2. **Java Console Server Application**

The client provides a graphical interface through which users authenticate and interact with stored files. The server manages file storage, metadata management, authentication, and command execution.

Communication between the client and server occurs using **TCP sockets** and a **custom binary packet protocol**. All commands and responses are transmitted using structured packets containing a command identifier and payload.

The design satisfies the following major system requirements:

- **SR-1:** Two-application architecture (client + server)
- **SR-3:** Custom packet communication protocol
- **SR-4:** Server state machine enforcing protocol validity
- **SR-5:** Large data transfer using chunked file transmission

Functional requirements implemented include authentication (FR-1–FR-3), file listing (FR-4–FR-5), upload (FR-6–FR-10), download (FR-11–FR-14), deletion (FR-15–FR-17), and logging (FR-18–FR-19).

System Architecture Description

DocNest uses a layered client-server architecture. The client application is responsible for user interaction and constructing requests.

The server handles authentication, storage, and command processing. All communication occurs using TCP connections between a client session and the server.

Core Architectural Components

Component	Description
Client UI Screens	JavaFX interface for login, file listing, upload, download, delete
Client Packet Builder	Serializes commands into packets

Client Session Manager	Manages connection and state
ServerMain	Accepts incoming TCP connections
ClientSession	Manages per-client session state
Packet Router	Routes packets to appropriate handlers
Command Handlers	Login, upload, download, delete
MetadataStore	Loads/saves metadata.json
FileStorageManager	Reads/writes file chunks
Logger	Writes timestamped logs

System Architecture Diagram

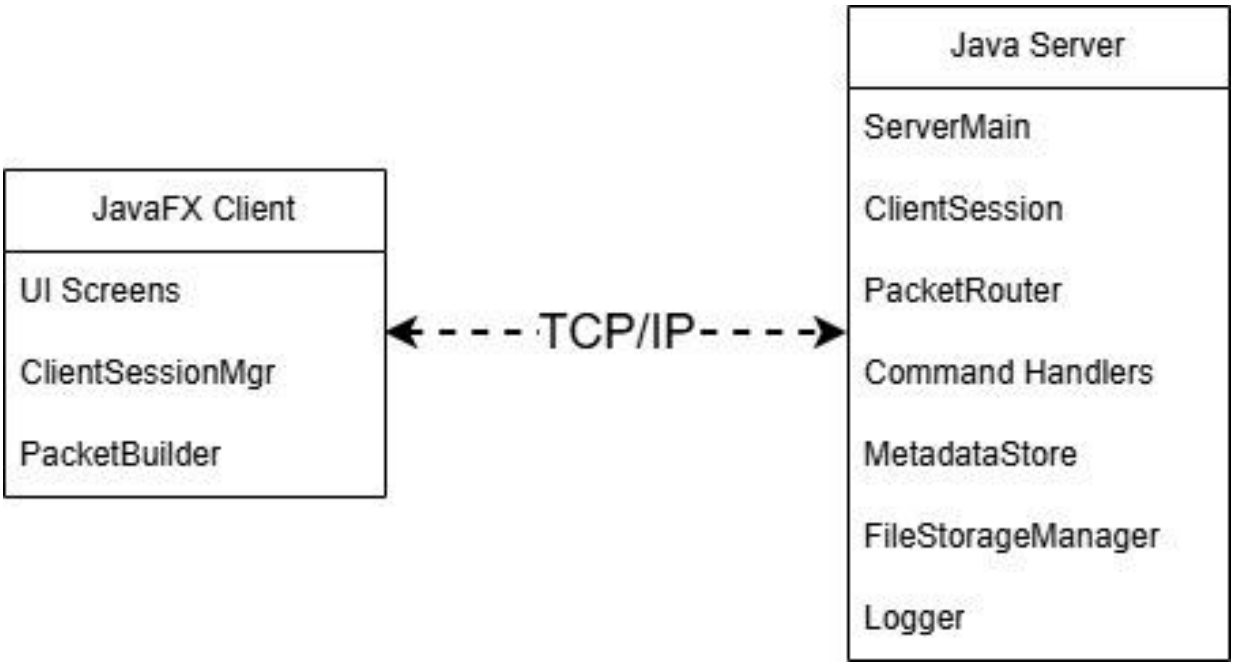


Figure 1: System Architecture Diagram

Data Packet Structure

All client-server communication uses a custom binary packet format. This packet structure implements SR-3 (Data Packet requirement).

Field	Type	Description
commandId	int	Identifies command type
payloadSize	int	Payload length
payload	byte[]	JSON or binary data

The payload format varies depending on whether the packet carries a control command or file transfer data. Payload types include:

- **JSON payloads** for control commands such as LOGIN or LIST_FILE
- **Binary chunks** for file transfer chunks

File transfers use a fixed chunk size of 4096 bytes to allow efficient streaming of large files. This protocol supports all major functional operations including login, file listing, upload, download, deletion, and logout.

Packet Layout Diagram

The diagram below shows the structure of the custom binary packet that we will use for this software. The payload size will be variable depending on if it is used for control command, response, or actual file/resource transfer.

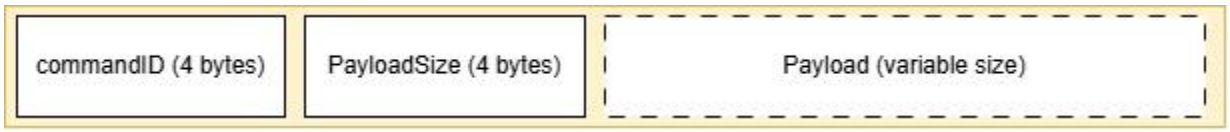


Figure 2: Packet Layout Diagram

Command Protocol

The following commands are supported by the DocNest protocol.

Command List

Command ID	Name	Direction	Payload
1	LOGIN	Client → Server	JSON
2	LOGIN_RESPONSE	Server → Client	JSON
3	LIST_FILES	Client → Server	JSON
4	LIST_FILES_RESPONSE	Server → Client	JSON
5	UPLOAD_INIT	Client → Server	JSON
6	UPLOAD_READY	Server → Client	JSON
7	UPLOAD_CHUNK	Client → Server	Binary
8	UPLOAD_COMPLETE	Client → Server	JSON
9	UPLOAD_RESULT	Server → Client	JSON
10	DOWNLOAD_INIT	Client → Server	JSON
11	DOWNLOAD_READY	Server → Client	JSON
12	DOWNLOAD_CHUNK	Server → Client	Binary
13	DOWNLOAD_COMPLETE	Server → Client	JSON
14	DELETE_FILE	Client → Server	JSON
15	DELETE_RESPONSE	Server → Client	JSON
16	LOGOUT	Client → Server	JSON
17	ERROR	Server → Client	JSON

Server Lifecycle State Machine

DocNest distinguishes between **two different types of system states**:

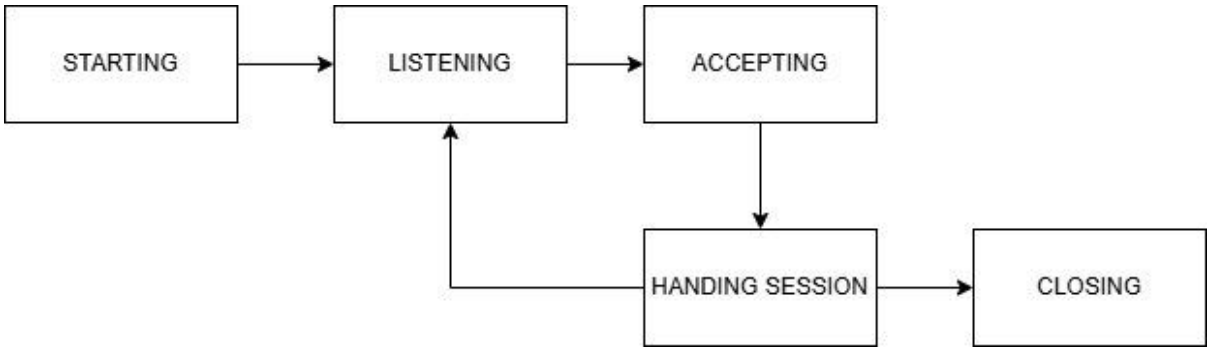
1. **Server lifecycle states**
2. **Session protocol states**

The server lifecycle state machine describes the global operational state of the server process.

States include:

- STARTING
- LISTENING
- ACCEPTING
- HANDLING_SESSION
- CLOSING

Server Lifecycle Diagram



The lifecycle states of the server and the state machine operate independently of that of the sessions it spins up to deal with authenticated clients. The lifecycle states of the individual session are governed by a separate protocol.

Session Protocol State Machine

The session protocol state machine governs the state of a single client connection.

States include:

- CONNECTED
- AUTHENTICATING
- READY
- TRANSFERRING
- CLOSING

These states enforce the communication protocol and prevent invalid command execution.

Allowed Commands per State

State	Allowed Commands
CONNECTED	LOGIN
READY	LIST_FILES, UPLOAD_INIT, DOWNLOAD_INIT, DELETE_FILE, LOGOUT
TRANSFERRING	UPLOAD_CHUNK, DOWNLOAD_CHUNK
CLOSING	none

Session Protocol State Machine Diagram

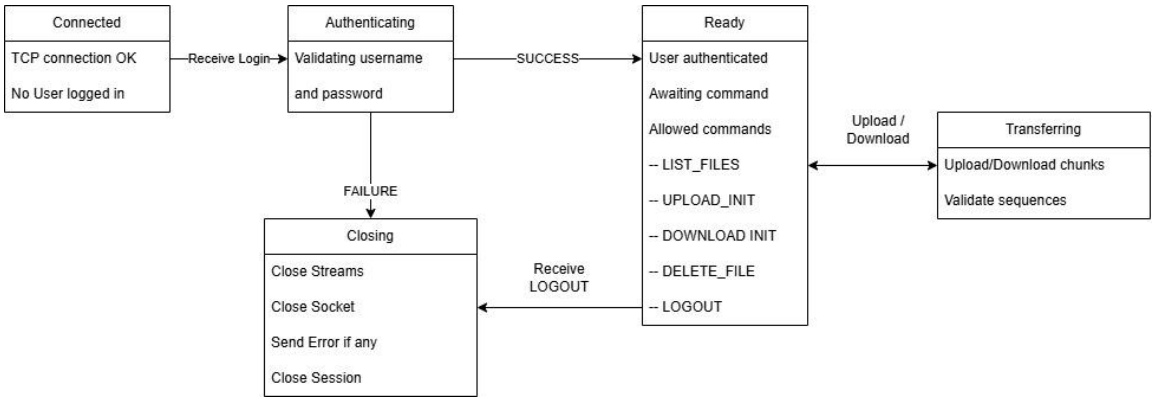


Figure 3: Session Protocol Lifecycle State Machine

Communication Flow

This section describes the typical interaction sequence between the client and server during common system operations. These interactions are governed by the communication protocol and session state machine described earlier.

The sequence diagrams illustrate how requests originating from the client application are processed by the server and how responses are returned through the network connection.

Login Interaction Sequence

The login operation establishes an authenticated session between the client and the server. The following sequence summarizes the interaction:

1. The client application establishes a TCP connection with the server.
2. The client sends a **LOGIN** request containing the user's authentication credentials.
3. The server validates the credentials against the registered user records.
4. Upon successful authentication, the server returns a **LOGIN_RESPONSE** packet.
5. The client requests the list of available files stored in the user's vault.
6. The user may then perform operations such as file upload, download, or deletion.
7. When the session ends, the client sends a **LOGOUT** command and the connection is closed.

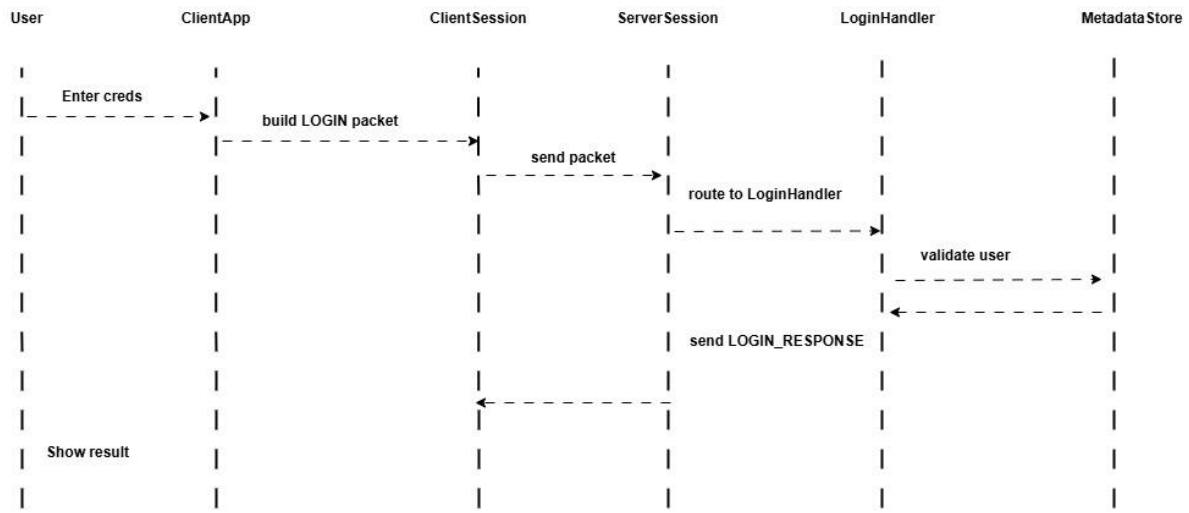


Figure 4: Sequence Diagram for Log In

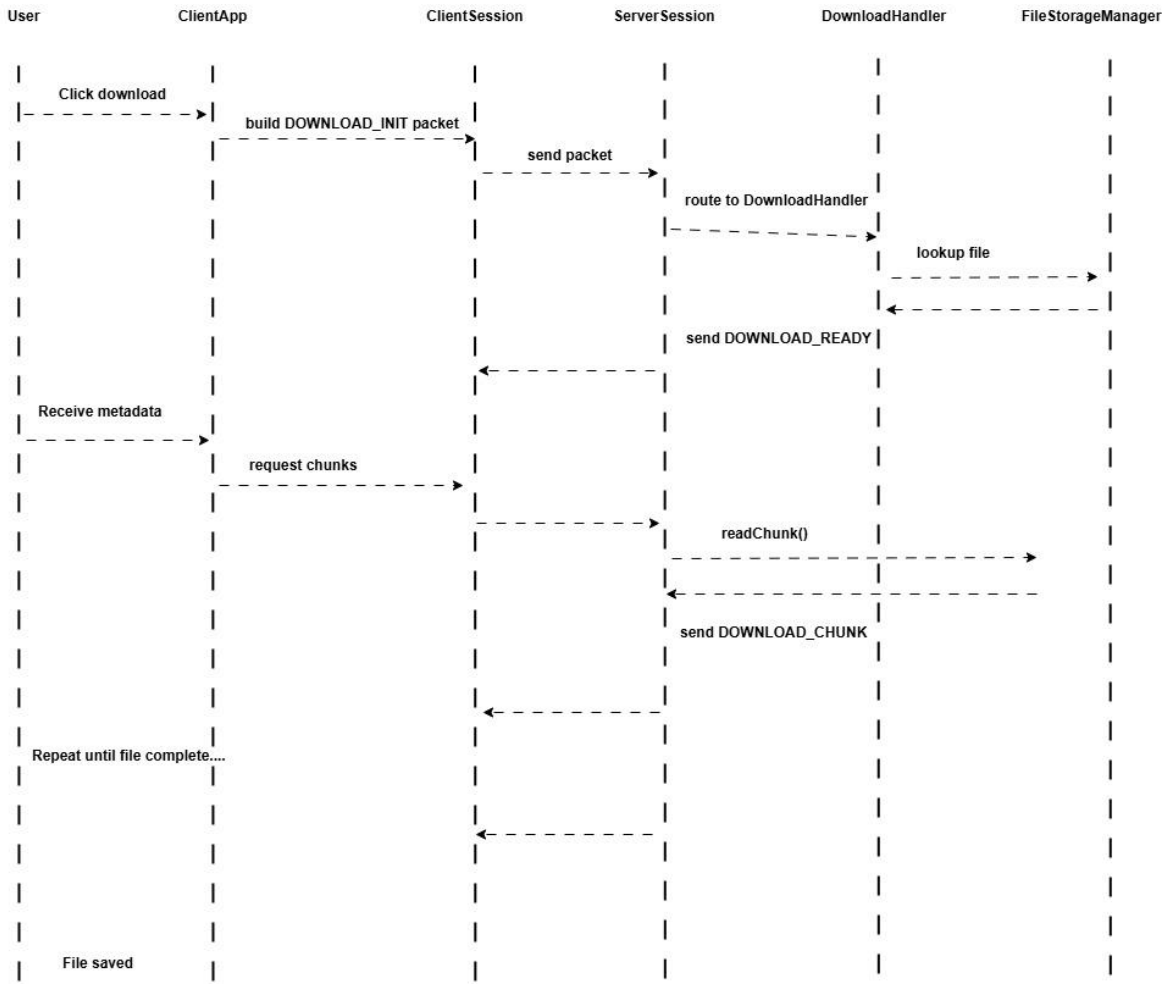


Figure 5: Sequence Diagram for Download File

File Download Interaction Sequence

The file download process involves multiple interactions between the client and server to retrieve file data in sequential chunks. This chunked transfer mechanism ensures efficient and reliable transmission of large files. The sequence of operations is as follows:

1. The user initiates a download request through the client application's user interface.
2. The client constructs a DOWNLOAD_INIT request and delegates packet creation to the ClientSession component.
3. The ClientSession transmits the request to the ServerSession through the active TCP connection.
4. The server identifies the request type and forwards it to the DownloadHandler for processing.
5. The DownloadHandler queries the FileStorageManager to locate the requested file and retrieve associated metadata.
6. The FileStorageManager returns information about the file including existence, size, and metadata.
7. The server sends a DOWNLOAD_READY response to the client indicating that the file transfer can begin.
8. The client receives the metadata and prepares to receive the file data.
9. The client requests the first file chunk from the server.
10. The server retrieves the next portion of the file using the FileStorageManager.
11. The retrieved data is packaged into a DOWNLOAD_CHUNK response packet.
12. The chunk is transmitted to the client.
13. The client receives the chunk and appends it to the local file buffer.
14. Steps 9–13 repeat until all file chunks have been transmitted.
15. After the final chunk is delivered, the server sends a DOWNLOAD_COMPLETE message indicating the end of the transfer.
16. The client assembles the received chunks and saves the file locally.

Client Design

Description

The DocNest client application provides a graphical interface that allows users to interact with documents stored on the server.

The client performs the following functions:

- Authenticate users
- Display stored files
- Upload new files
- Download existing files
- Delete files
- Display metadata

The client application is implemented using **JavaFX** and communicates with the server using TCP sockets.

The client architecture separates **user interface components from networking logic** to maintain modular design and satisfy maintainability requirements.

Client Software Architecture Description

The client architecture consists of three primary components:

- User Interface
- ClientSessionManager
- PacketBuilder

User interactions within the graphical interface generate requests that are transmitted to the server through the session manager.

Client Architecture Diagram

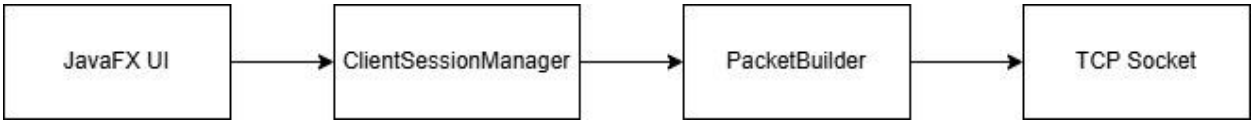


Figure 6: Client Architecture Diagram

The client architecture is composed of the JavaFX user interface, the ClientSessionManager responsible for maintaining the server connection, and the PacketBuilder which constructs protocol-compliant packets that are transmitted through the TCP socket.

Client Class Diagram

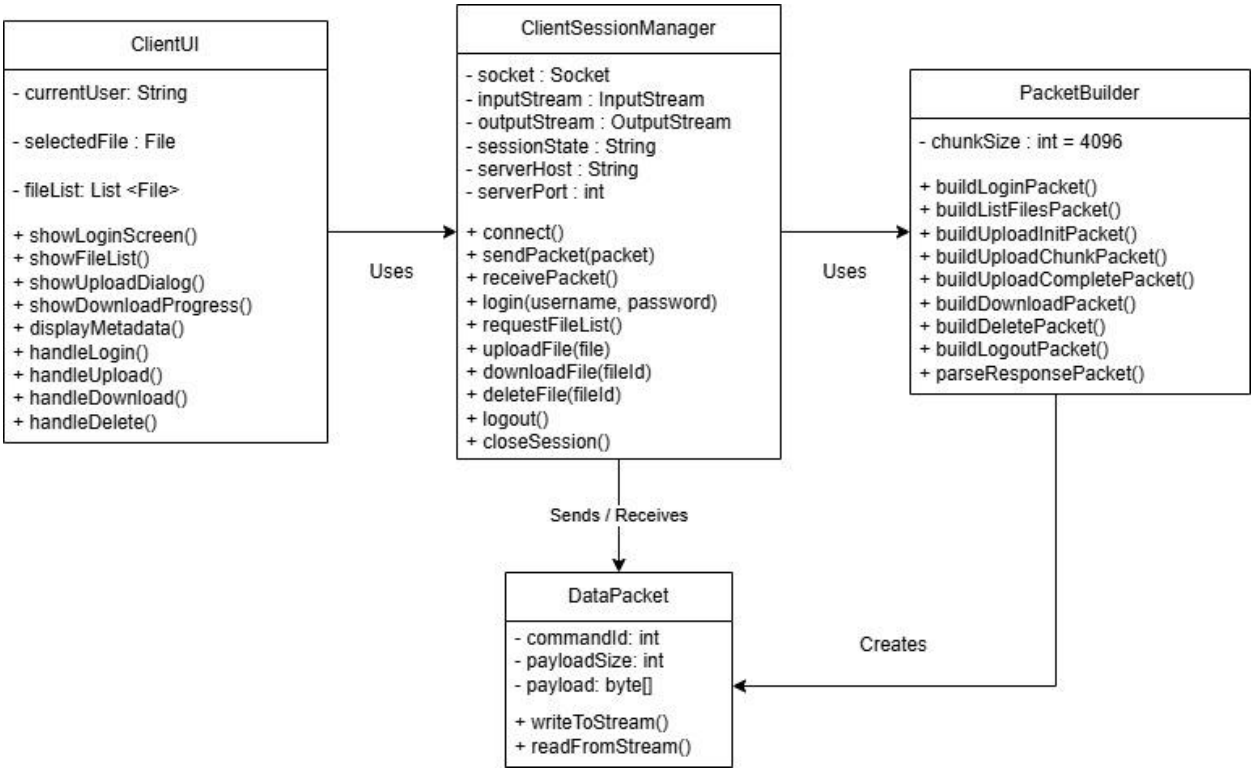


Figure 7: Client Class Diagram

The client class diagram illustrates the relationships between the primary classes used in the client application. The ClientUI handles user interaction, while the ClientSessionManager manages the network session and communicates with the server. The PacketBuilder class constructs the DataPacket objects that are transmitted over the TCP connection.

ClientSessionManager – Moutushi Sarkar

Description

The **ClientSessionManager** class is responsible for managing the network communication between the client application and the server. It maintains the active TCP connection, manages the client session state, and handles the sending and receiving of packets according to the DocNest communication protocol.

This class acts as the primary interface between the user interface layer and the network layer. All client requests generated by the graphical interface are routed through the ClientSessionManager before being transmitted to the server.

Functional Design

The ClientSessionManager performs several key responsibilities within the client application.

These responsibilities include:

- Establishing and maintaining the TCP connection with the server

- Managing the lifecycle of the client session
- Sending request packets to the server
- Receiving and interpreting response packets from the server
- Handling connection termination and cleanup procedures

The class ensures that all communication follows the protocol defined by the system packet structure and session state machine.

Key Methods

The primary methods implemented within this class include:

- **connect()** establishes a TCP connection to the server.
- **sendPacket()** transmits a DataPacket object to the server.
- **receivePacket()** listens for and processes responses from the server.
- **closeSession()** safely terminates the network connection and releases resources.

PacketBuilder – Moutushi Sarkar

Description

The **PacketBuilder** class is responsible for constructing the packets used for communication between the client and server. It encapsulates the logic required to assemble protocol-compliant DataPacket objects from application-level commands and data.

By isolating packet construction within a dedicated class, the design ensures that communication formatting remains consistent throughout the application and simplifies maintenance of the network protocol.

Functional Design

The PacketBuilder handles the transformation of client requests into properly structured packet objects.

Its responsibilities include:

- Serializing command data into JSON format for control messages
- Packaging binary data when transmitting file chunks
- Constructing DataPacket objects according to the protocol specification
- Ensuring packet fields such as command identifiers and payload sizes are correctly formatted

This separation of concerns allows the ClientSessionManager to focus solely on network transmission while delegating packet construction to the PacketBuilder.

Key Methods

The PacketBuilder provides several methods used to construct packets for the various system operations:

- buildLoginPacket()
- buildUploadPacket()
- buildDownloadPacket()
- buildDeletePacket()

Each method constructs a packet corresponding to a specific command defined in the system protocol.

Client Data Flow

This section describes the interaction between the main client components during typical file operations. The client application follows a layered interaction model where the user interface initiates requests that are processed through the network management components before being transmitted to the server.

Upload Operation Flow

The following sequence illustrates the typical process for uploading a file from the client to the server:

1. The user selects a file to upload through the graphical user interface.
2. The user interface sends an upload request to the **ClientSessionManager**.
3. The **PacketBuilder** constructs an UPLOAD_INIT packet containing file metadata.
4. The ClientSessionManager transmits the packet to the server through the TCP connection.
5. The server responds with an UPLOAD_READY message indicating that it is prepared to receive file data.
6. The client begins transmitting the file as a sequence of fixed-size data chunks until the entire file has been transferred.

This process allows large files to be transferred reliably using the chunked data transfer mechanism defined by the communication protocol.

Sequence Diagram Description

The accompanying sequence diagram illustrates the interaction between the major client components during a file upload operation.

The user initiates the upload request through the **JavaFX user interface**, which delegates the request to the **ClientSessionManager**. The session manager then utilizes the **PacketBuilder** to construct the appropriate protocol packet and sends the request to the server. Server responses are received through the session manager and forwarded back to the user interface for display and progress updates.

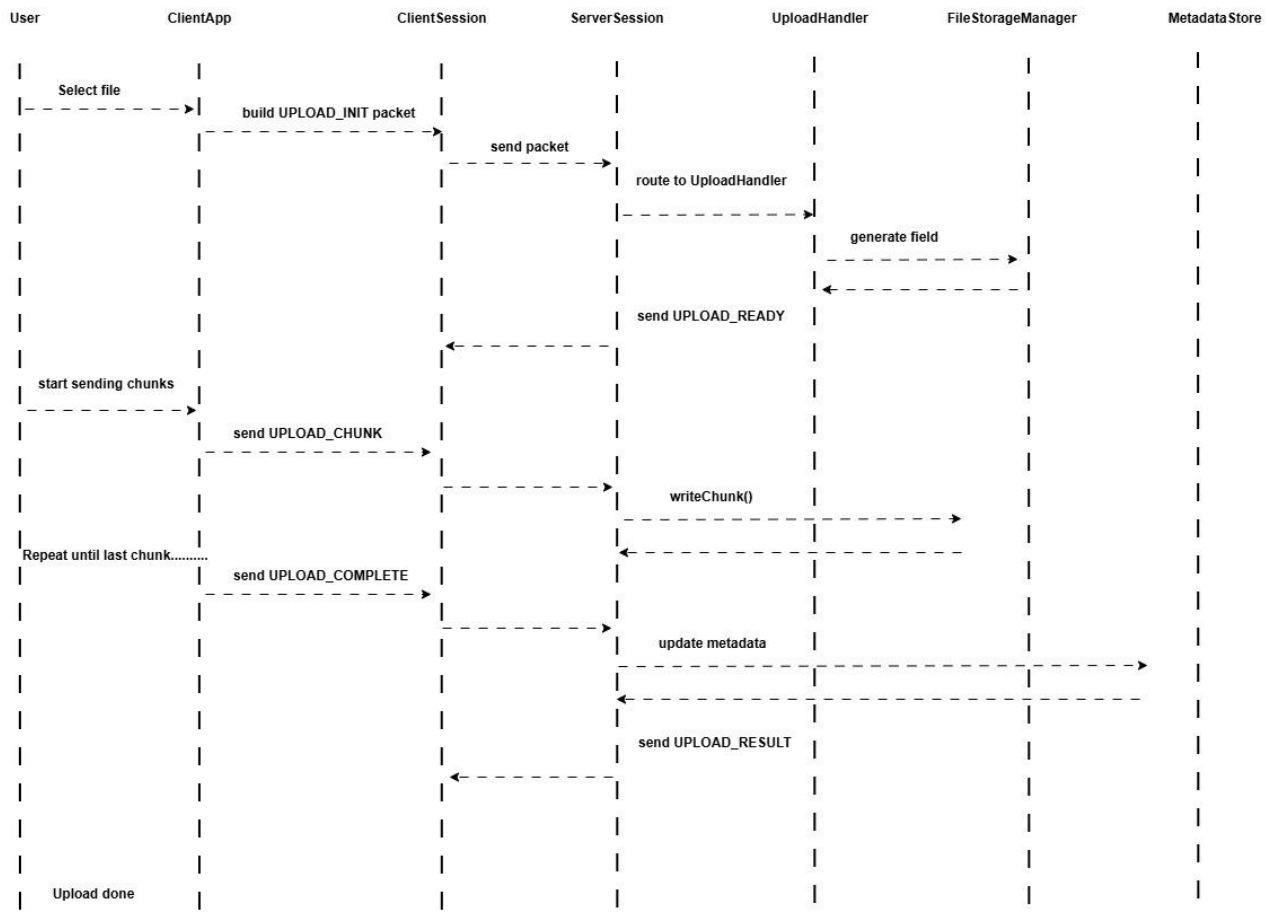


Figure 8: Sequence diagram for Upload Use case

Graphical User Interface Design Description

The client interface is designed to be intuitive and user-friendly, satisfying the usability requirements defined in the Software Requirements Specification. The graphical interface provides a streamlined environment for interacting with the document storage system and supports all core user operations including authentication and file management.

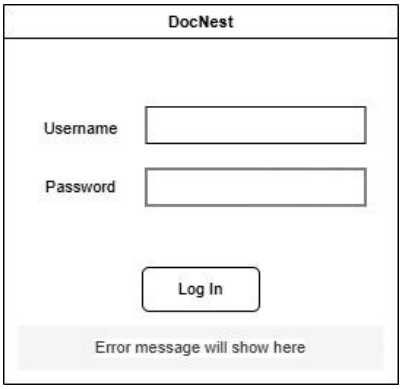


Figure 9: Login Screen Wireframe

- **Login Screen** – Allows users to authenticate using their credentials.

- **File List Screen** – Displays the list of files stored in the user's vault.

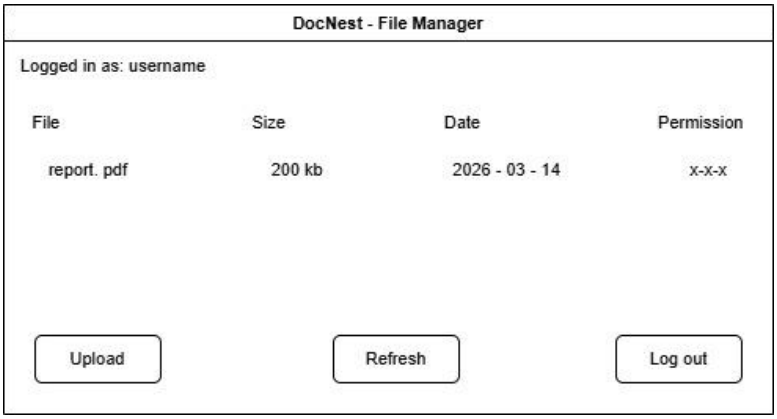


Figure 10: File List Screen WireFrame

- **Upload Dialog** – Enables users to select and upload files to the server.

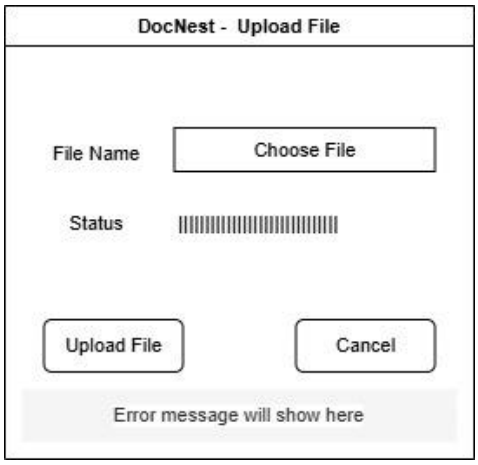


Figure 11: Upload File Wireframe

- **Download Progress Dialog** – Displays the progress of ongoing download operations.

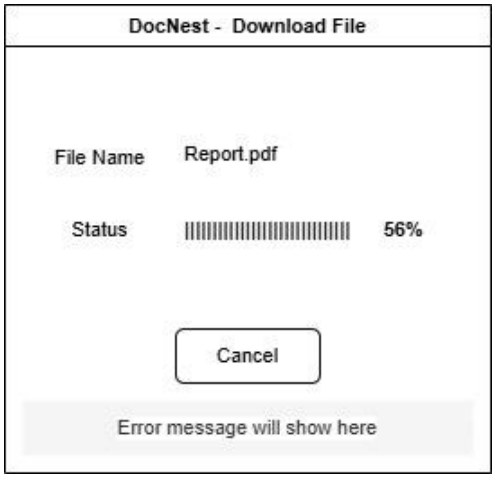


Figure 12: Download File Wireframe

- **Metadata View** – Displays detailed information about selected files.

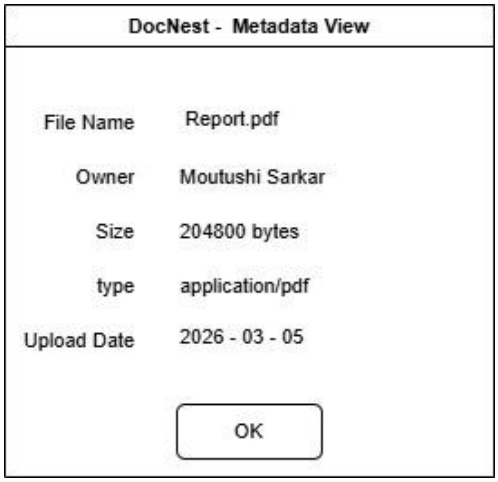


Figure 13: Metadata view wireframe

These interface components provide clear feedback to the user and allow file operations to be performed in a simple and efficient manner.

Server Design

Description

The DocNest server manages authentication, file storage, metadata management, and logging. It accepts TCP connections from clients and processes commands using the defined packet protocol.

The server enforces authentication before allowing any file operations and manages file storage within isolated user vault directories.

Major server responsibilities include:

- Authenticating users
- Managing metadata storage
- Handling file uploads and downloads
- Processing client commands
- Logging system activity

Server Software Architecture Description

The DocNest server is designed using a modular architecture in which each component is responsible for a specific aspect of server functionality. This modular structure improves maintainability, simplifies debugging, and ensures that the responsibilities of connection management, command processing, file storage, and logging remain clearly separated.

The primary server components include:

- ServerMain
- ClientSession
- PacketRouter
- Command Handlers
- MetadataStore
- FileStorageManager
- Logger

The **ServerMain** component acts as the entry point for the server application. It initializes the server environment, loads configuration data such as user credentials and metadata, and opens the server socket to listen for incoming client connections.

When a client connects, the server creates a **ClientSession** instance to manage the communication with that specific client. Each ClientSession maintains the session state, reads incoming packets from the network, and sends responses back to the client.

Incoming packets received by the ClientSession are forwarded to the **PacketRouter**, which determines the type of command contained in the packet. The PacketRouter then delegates processing to the appropriate **command handler**, such as login, upload, download, or delete operations.

These command handlers interact with other server components to complete their tasks. The **MetadataStore** manages the metadata associated with stored files, including loading and updating the metadata.json file. The **FileStorageManager** handles the physical storage of file data, including writing uploaded file chunks, retrieving files for download, and deleting files from the vault directory.

Throughout the execution of server operations, the **Logger** component records system events, errors, and operational messages to a persistent log file. This logging functionality supports debugging, monitoring, and system maintenance.

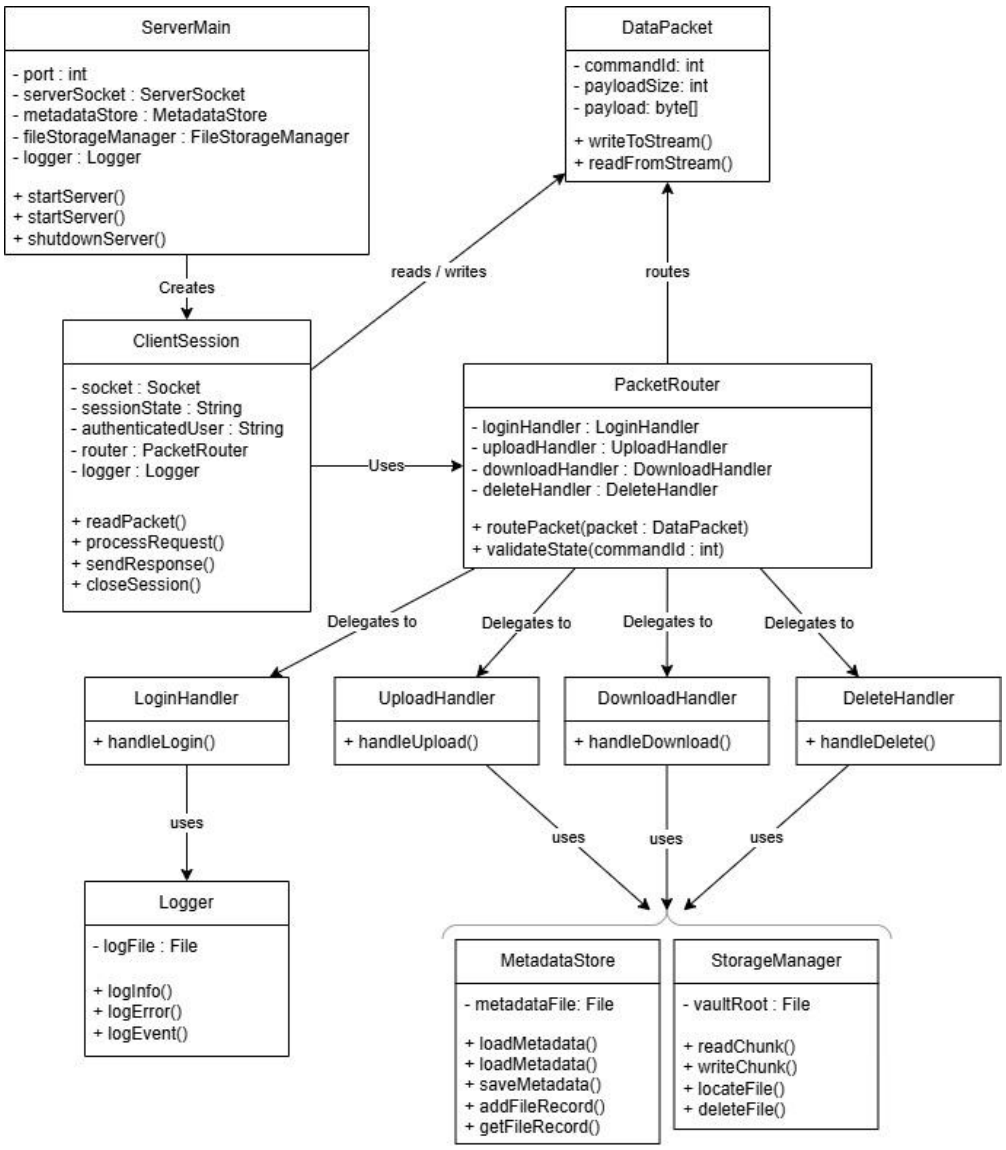


Figure 14: Server Class Diagram

The server class diagram above illustrates the relationships between the primary server components and how they collaborate to process client requests.

User Interface Design Description

The server uses a **console-based interface** that provides system status and logging output.

The console displays events such as:

- server startup
- client connections
- authentication events
- file operations
- system errors

Example Server Console Output

[INFO] DocNest Server Starting...

[INFO] Listening on port 5000

[INFO] Client connected: 192.168.1.10

[INFO] User authenticated: moutushi

[INFO] Upload started: report.pdf

[INFO] Upload completed: report.pdf

[INFO] Download request: notes.txt

[INFO] File deleted: draft.docx

[INFO] Client disconnected

Server Modules

ServerMain – Moutushi Sarkar

Description

ServerMain initializes the server environment and listens for incoming client connections.

Functional Design

Responsibilities include:

- Loading metadata
- Loading user credentials
- Initializing vault directories
- Starting ServerSocket
- Accepting incoming connections

ClientSession – Moutushi Sarkar

Description

ClientSession represents a single client connection and maintains the session state for that client.

Functional Design

Responsibilities include:

- Maintaining session protocol state
- Reading incoming packets
- Forwarding packets to PacketRouter
- Closing connections when necessary

PacketRouter - Moutushi Sarkar

Description

PacketRouter routes incoming packets to the appropriate command handler.

Functional Design

Processing flow:

receive packet

extract commandId

switch(commandId)

call appropriate handler

return response

MetadataStore - Moutushi Sarkar

Description

MetadataStore manages the metadata.json file that tracks information about stored files.

Functional Design

Responsibilities include:

- Loading metadata during server startup
- Updating metadata after uploads
- Removing entries after deletions

FileStorageManager - Moutushi Sarkar

Description

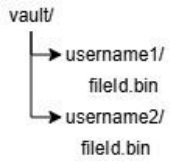
FileStorageManager handles the reading and writing of file data to disk.

Functional Design

Responsibilities include:

- Writing uploaded file chunks
- Reading files for download
- Deleting stored files

File storage structure:



Logger - Moutushi Sarkar

Description

The Logger component records system events to a persistent log file.

Functional Design

Responsibilities include:

- Recording timestamps
- Logging authentication events
- Logging file operations
- Logging system errors

Log file location:

logs/server.log

Data Structures

The following JSON structures would be implemented for the server to capture the necessary data and keep track of the resources and communications with the client.

FileRecord

This is the metadata associated with each file. This will be stored as JSON (as opposed to some database structure for simplicity)

json

```
{
  "id": "uuid",
  "owner": "username",
  "name": "resume.pdf",
  "size": 204800,
  "mime": "application/pdf",
  "uploadedAt": "2026-03-05T22:04:00"
}
```

UserRecord

This will also be stored in a separate JSON file. This information will be used to validate authentication. For the demonstration purposes, we will hardcode registered users.

json

```
{
  "username": "moutushi",
  "password": "1234"
}
```

DataPacket Class

This would be the class that will be structuring the data being transferred.

Fields:

- int commandId
- int payloadSize
- byte[] payload

Methods:

- writeToStream()
- readFromStream()

Core Algorithms

This section describes the operational flow of the main file management and authentication processes supported by the DocNest system. The descriptions below illustrate the logical sequence of actions performed by the server when handling client requests. These descriptions are not intended to represent strict pseudocode; rather, they provide a conceptual overview of how the system processes each type of operation during a client session.

Authentication Process

The authentication process is responsible for validating user credentials before any file operations can occur. When a client first connects to the server, the session enters the **CONNECTED** state. The client must successfully authenticate before the session transitions to the **READY** state where file operations are permitted.

Authentication proceeds as follows:

1. The server receives a **LOGIN** packet from the client containing the username and password encoded as a JSON payload.
2. The server parses the JSON payload to extract the submitted credentials.
3. The credentials are validated against the stored user records in the **users.json** file.
4. If the credentials are valid, the server updates the session state to **READY** and sends a **LOGIN_RESPONSE** packet confirming successful authentication.
5. If authentication fails, the server returns an **ERROR** packet indicating authentication failure and terminates the session.

This process ensures that only authenticated users are permitted to interact with the file storage system.

File Upload Process

The file upload process enables a client to transfer a file to the server in a controlled and reliable manner. Because files may be large, they are transmitted in fixed-size chunks rather than as a single payload.

The upload process operates as follows:

1. The client sends an **UPLOAD_INIT** packet containing file metadata such as filename and size.
2. Upon receiving the request, the server generates a unique **fileId** (typically a UUID) that will be used to identify the stored file.
3. The server responds with an **UPLOAD_READY** packet indicating that it is prepared to receive file data.
4. The client begins transmitting the file as a sequence of **UPLOAD_CHUNK** packets, each containing a fixed-size block of binary data (typically 4096 bytes).

5. As each chunk is received, the server writes the data sequentially to a file in the user's vault directory.
6. When all chunks have been transmitted, the client sends an **UPLOAD_COMPLETE** packet indicating that the transfer has finished.
7. The server finalizes the file storage process and updates the **metadata.json** file to include the new file record.

This chunked transfer mechanism ensures reliable handling of large files while maintaining protocol consistency.

File Download Process

The file download process allows a client to retrieve a stored file from the server. Similar to uploads, file transfers occur in chunks to support efficient streaming and avoid large memory usage.

The download sequence proceeds as follows:

1. The client sends a **DOWNLOAD_INIT** request specifying the **fileId** of the file to retrieve.
2. The server verifies that the file exists and that the requesting user has permission to access
3. If the file is valid, the server sends a **DOWNLOAD_READY** response containing metadata such as file size and type.
4. The server reads the stored file and transmits it to the client in a sequence of **DOWNLOAD_CHUNK** packets, each containing a block of binary data.
5. After all file data has been transmitted, the server sends a **DOWNLOAD_COMPLETE** packet indicating the end of the transfer.

The client reconstructs the file locally by assembling the received chunks in sequence.

File Deletion Process

The deletion process allows a user to permanently remove a stored file from the server.

The deletion sequence occurs as follows:

1. The client sends a **DELETE_FILE** request specifying the **fileId** of the file to remove.
2. The server verifies that the file exists and that the requesting user is authorized to delete it.
3. If the file is valid, the server deletes the corresponding file from the user's vault directory.
4. The server updates the **metadata.json** file to remove the file's metadata entry.
5. A **DELETE_RESPONSE** packet is sent to the client confirming successful deletion.

This process ensures that both the physical file and its associated metadata are removed from the system.

Error Handling

The DocNest system implements a structured error handling mechanism to ensure robust communication between the client and server. When an error occurs during command processing, the server responds with a standardized **ERROR packet** containing a JSON payload that describes the issue.

Error Packet Format

This structured format allows the client application to interpret the error and provide appropriate feedback to the user.

Example Error Packet Format

```
{
 "code": "INVALID_STATE",   "message": "Command not allowed in current session state.",
  "details": null
}
```

The **code** field identifies the specific error condition, while the **message** field provides a human-readable description of the problem. The optional **details** field may contain additional diagnostic information when applicable.

Error Categories

The server defines several error categories to handle different types of protocol violations and runtime failures. Some errors require the session to be terminated to maintain protocol integrity, while others allow the session to continue.

Error Code	Description	Close Session
AUTH_FAILED	Authentication failed due to invalid credentials	Yes
INVALID_STATE	Command issued in an invalid session state	Yes
INVALID_COMMAND	Unrecognized command identifier	Yes
FILE_NOT_FOUND	Requested file does not exist	No
CHUNK_OUT_OF_ORDER	File transfer chunks received in an incorrect sequence	Yes
BAD_REQUEST	Malformed packet or invalid request format	Yes
INTERNAL_ERROR	Unexpected server-side failure	Yes

Error Logging

All errors generated by the server are recorded in the system log. Each log entry includes a timestamp, the error code, and relevant contextual information. Logging errors ensures that system failures can be diagnosed and supports troubleshooting during development and maintenance.

Security Design

The DocNest system incorporates several security mechanisms to ensure that user data is protected and that only authorized operations are permitted during a client session.

Credential Storage

User credentials are maintained in a server-side configuration file named **users.json**. This file stores the usernames and associated passwords required for authentication. For the purposes of this course project, storing passwords in plain text is acceptable; however, the design allows for the optional use of **SHA-256 hashing** to improve security. If hashing is enabled, passwords are stored as hashed values rather than readable text, reducing the risk of credential exposure.

Vault Isolation

To ensure that users cannot access each other's files, the server organizes stored documents using a **vault directory structure**. Each authenticated user has a dedicated folder where their uploaded files are stored.

Example storage structure: vault/<username>/<fileId>.bin

This design ensures that files are logically separated by user and helps enforce access control when processing file requests.

Session Enforcement

All file operations require a successfully authenticated session. The server enforces this rule using the session state machine defined in the communication protocol. Commands such as file listing, upload, download, or deletion are only permitted after the client has completed the **LOGIN** operation. The session state machine prevents invalid command transitions and ensures that unauthorized operations cannot occur before authentication is complete.

Traceability to Requirements

This section illustrates how the major design components of the DocNest system correspond to the functional and non-functional requirements defined in the Software Requirements Specification. Establishing traceability ensures that each requirement is addressed by a specific architectural component within the system design. This mapping also helps verify that the implementation will satisfy the required system functionality, performance, security, and reliability constraints.

Design Component	Related FRs	Related NFRs
Packet Protocol	FR-1–FR-17	NFR-1, NFR-2, NFR-3
State Machines	FR-1–FR-17	NFR-3, NFR-7
MetadataStore	FR-4, FR-10, FR-17	NFR-4, NFR-9
FileStorageManager	FR-6–FR-14	NFR-1, NFR-2
Logger	FR-18–FR-19	NFR-3, NFR-9
Client UI	FR-1–FR-17	NFR-5, NFR-6
Authentication	FR-1–FR-3	NFR-7, NFR-8